CLOUD COMPUTING

Web Services with SOAP and Azure Integrations

SMART EV CHARGING FINDER

A Distributed SOAP-Based Electric Vehicle Charging Recommendation Service

Name	Tejovanth S H
Register Number	2025178023
Programme	II MCA (Regular)
Subject	Cloud Computing

Technologies Used

Java 21 Spring Boot SOAP REST
Open Charge Map Open-Meteo Microsoft Azure

1. Abstract

The Smart EV Charging Finder is a distributed web application developed using Java, Spring Boot, SOAP and Microsoft Azure. The system helps an electric vehicle user find a nearby charging station compatible with the required charging connector.

The application consists of three independent Spring Boot services: a Recommendation Server, a Station Server and a Service Registry. The Recommendation Server accepts a human-readable location and uses the Open-Meteo Geocoding API to obtain geographical coordinates. It then discovers the Station Server through the registry and communicates with it using SOAP. The Station Server retrieves charging station information dynamically from the Open Charge Map API using REST.

The services are deployed on an Ubuntu Linux virtual machine in Microsoft Azure. The project demonstrates SOAP Web Services, REST API integration, service discovery and cloud deployment in a single distributed application.

2. Introduction

Finding a suitable charging station is an important requirement for electric vehicle users, especially when different vehicles support different charging connectors. This project implements a simple distributed charging station finder that accepts a user's location and connector requirement and provides a suitable nearby station.

The application does not maintain a fixed list of charging stations. Instead, station information is obtained dynamically from the Open Charge Map API. A human-readable location such as "Guindy, Chennai" is converted into geographical coordinates using the Open-Meteo Geocoding API.

The project demonstrates both SOAP and REST Web Services. SOAP is used for communication between the Recommendation Server and Station Server, while REST is used for external APIs and service discovery.

2.1 Objectives

- Develop a distributed application using Java and Spring Boot.
- Implement a SOAP-based Web Service.
- Demonstrate service discovery using a service registry.
- Integrate real external REST APIs.
- Dynamically resolve user locations.
- Recommend a compatible operational charging station.
- Deploy the application on Microsoft Azure.

3. System Architecture

The application follows a distributed service-oriented architecture. The three Spring Boot services have separate responsibilities, while external APIs provide location and charging station information.

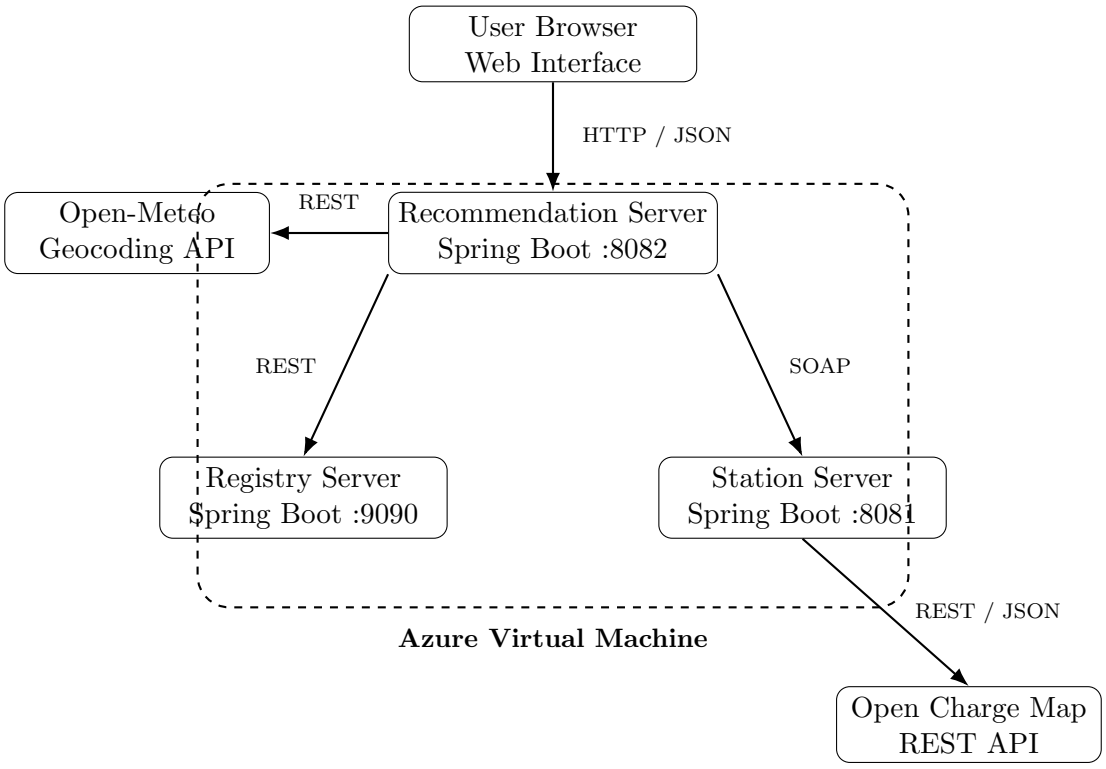


Figure 1: System Architecture

The browser communicates with the Recommendation Server, which coordinates the complete request. It resolves the user's location, discovers the Station Server through the Registry Server and invokes the Station Server using SOAP.

The Station Server retrieves charging station information from Open Charge Map and converts the external JSON response into the application's SOAP model. The Registry Server provides the Station Server endpoint to the Recommendation Server.

The three Spring Boot services are hosted on the Azure virtual machine, while the external APIs are accessed through the Internet.

4. Data Flow

The complete request processing flow is:

1. The user enters a location, battery information and connector type.
2. The browser sends the request to the Recommendation Server.
3. The location is sent to the Open-Meteo Geocoding API.
4. Open-Meteo returns latitude and longitude.
5. The Recommendation Server queries the Registry Server for `EVStationService`.
6. The Registry returns the Station Server endpoint.
7. The Recommendation Server sends a SOAP request containing the coordinates and search radius.
8. The Station Server queries Open Charge Map using REST.
9. The returned JSON is mapped into `Station` and `ChargingPort` objects.
10. Compatible operational stations are filtered and the nearest suitable station is selected.
11. The recommendation is returned to the browser as JSON.

The overall flow can be summarized as:

Browser → Recommendation Server → Open-Meteo → Registry → Station Server → Open Charge Map

The system does not use hardcoded charging station coordinates or station records.

5. External APIs

5.1 Open-Meteo Geocoding API

Open-Meteo is used to convert a human-readable location into geographical coordinates. For example, the user can enter:

```
Guindy, Chennai
```

The API returns geographical information including latitude and longitude. These coordinates are used for the charging station search.

This allows the application to accept normal location names instead of requiring the user to manually provide coordinates.

5.2 Open Charge Map API

Open Charge Map provides the charging station information used by the application. The Station Server sends the geographical coordinates, search radius and result parameters to the API.

The returned information can include:

- Station ID and name
- Address and coordinates
- Charging connections
- Connector type
- Charging power
- Operational status

The Station Server converts the returned JSON into the internal SOAP model. Multiple connections belonging to a station can be represented as multiple charging ports.

The external API is therefore an essential part of the application rather than an optional feature.

6. SOAP Web Service

The Station Server provides the main SOAP Web Service of the application. The service is described using a WSDL and XML schema.

The SOAP endpoint is:

```
http://<server>:8081/ws
```

The service accepts:

- Latitude
- Longitude
- Search radius in kilometres

The response contains charging stations with information such as:

- Station ID and name
- Location and coordinates

- Operational status
- Charging ports
- Connector type
- Charging power

SOAP acts as an abstraction layer between the Recommendation Server and the external charging station API. The Recommendation Server therefore receives a structured SOAP response instead of directly processing the external JSON format.

This demonstrates the use of SOAP for communication between independent application services.

7. Service Registry

A lightweight Registry Server is included to demonstrate service discovery. It runs on port 9090 and stores service endpoint information.

The Station Server is registered under the service name:

`EVStationService`

The registry provides two main REST operations:

`Endpoint`	Purpose
`POST /registry/register`	Register a service endpoint
`GET /registry/find/{serviceName}`	Discover a service

The Recommendation Server queries the registry before making the SOAP request. This demonstrates service discovery without directly depending on the Station Server endpoint.

The registry uses an in-memory structure because persistent service management is outside the scope of this mini project.

8. Recommendation Logic

After receiving the station data through SOAP, the Recommendation Server processes the available stations.

The recommendation process is:

1. Remove stations that are not operational.
2. Check compatibility with the requested connector type.
3. Calculate the distance from the user's location.
4. Select the nearest compatible station.

Distance is calculated using the Haversine formula:

$$d = 2R \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$$

where R represents the approximate radius of Earth.

The final response contains the station name, location, distance, connector type, charging power and operational status.

If no suitable station is available, the application returns an appropriate error instead of generating artificial station information.

9. Tools and Technologies

The following tools and technologies were used during development, testing and deployment.

Tool / Technology	Purpose
Java 21	Application development
Spring Boot	Development of the distributed services
Spring Web Services	SOAP Web Service implementation
Maven	Build and dependency management
Open-Meteo	Location geocoding
Open Charge Map	Charging station data
Microsoft Azure	Cloud deployment
Ubuntu Linux	Server operating system
SSH	Remote access to the Azure VM
WinSCP	Transfer of project files to the VM
PowerShell / cURL	Service and API requests
Web Browser	User interface

The three services were developed as separate Maven projects. Maven was used to build and package the applications. WinSCP was used to transfer the project files to the Azure virtual machine, while SSH was used for remote administration and running the services.

10. Azure Deployment

The application was deployed on an Ubuntu Linux virtual machine using Microsoft Azure. Java 21 and Maven were installed on the virtual machine, and the three Spring Boot services were executed independently.

Service	Port
Recommendation Server	8082
Station Server	8081
Registry Server	9090

Port 8082 provides the public entry point to the web application. The Station Server and Registry Server are used for internal communication within the virtual machine.

The project files were uploaded using WinSCP and the services were built and started remotely using SSH. The Open Charge Map API key is stored separately in the Station Server configuration and is not included in the source code or this report.

The Azure deployment demonstrates how multiple independent Spring Boot services can operate together within a cloud-hosted environment.

11. Implementation and Result

The three Spring Boot services were successfully implemented, built and deployed. The Station Server exposes a SOAP WSDL and retrieves charging station information dynamically from Open Charge Map.

The Recommendation Server resolves user locations, discovers the Station Server through the registry, invokes the SOAP service and processes the returned station information.

The complete workflow was successfully demonstrated through the web interface. For a valid location and compatible connector, the system returns a nearby charging station with its location, distance, connector type, charging power and operational status.

The application was also accessed through the public Azure-hosted Recommendation Server, demonstrating successful cloud deployment of the distributed application.

12. Limitations

The charging station results depend on the coverage and accuracy of the Open Charge Map dataset. Therefore, some locations may not return a station even when charging infrastructure exists nearby.

The operational status provided by the external API should not be considered a guaranteed real-time indication of charger occupancy.

The Registry Server uses in-memory registration, so the service registration needs to be restored after a registry restart.

These limitations are acceptable for the scope of the mini project.

13. Conclusion

The Smart EV Charging Finder demonstrates a distributed Web Services application using Java, Spring Boot, SOAP, REST APIs and Microsoft Azure.

The Recommendation Server coordinates the application, the Station Server provides charging station information through SOAP, and the Registry Server demonstrates service discovery. Open-Meteo dynamically resolves user locations, while Open Charge Map provides the actual charging station data.

The successful Azure deployment demonstrates how the independent services can be hosted in a cloud environment. Overall, the project provides a practical implementation of SOAP communication, REST API integration, service discovery and cloud deployment.

References

1. Open Charge Map. https://openchargemap.org/
2. Open-Meteo. https://open-meteo.com/
3. Spring Boot. https://spring.io/projects/spring-boot
4. Spring Web Services. https://spring.io/projects/spring-ws
5. Microsoft Azure Virtual Machines. https://learn.microsoft.com/azure/virtual-machines/